

Applied Type Theory

Лекция 3: Просто типизированное λ -исчисление

Воронов Михаил Сергеевич

ВМК МГУ

Осень 2025

- 1 Мотивация и исторические перспективы теории типов
- 2 Просто типизированное λ -исчисление: синтаксис и правила типизации
- 3 Метатеоретические свойства и основные результаты

- 1 Мотивация и исторические перспективы теории типов
- 2 Просто типизированное λ -исчисление: синтаксис и правила типизации
- 3 Метатеоретические свойства и основные результаты

Ограничения нетипизированного λ -исчисления

- ❶ **Самоприменение:** $(\lambda x. x x) (\lambda x. x x)$ — бесконечное вычисление
- ❷ **Неподвижные точки:** $Y \equiv \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$ — для любого терма F существует X с $F X =_{\beta} X$
- ❸ **Нетерминация:** не все термы имеют нормальную форму
- ❹ **Парадоксы:** возможны формы самоссылки и наивные логические кодировки, которые при интерпретации в логике приводят к парадоксам (Рассела и др.)

Проблема: в нетипизированном λ -исчислении нельзя гарантировать завершение вычислений или логическую непротиворечивость.

Понятие типа в информатике

Существует несколько распространённых определений понятия типа (Parnas, Shore, Weiss, 1976):

- **Тип** — это синтаксическая метка, ассоциированная с переменной (синтаксический).
- **Тип** — это некоторая композиция более примитивных типов (обычно, машинных типов) (репрезентативный).
- **Тип** — это некоторая композиция более примитивных типов и набор операторов для манипулирования этими представлениями (репрезентативный и поведенческий).
- **Тип** — это множество всех возможных значений, которые может принимать переменная (классы эквивалентности) (ориентированный на значения).
- **Тип** — это множество всех возможных значений, которые может принимать переменная и множество функций, которые могут быть применены к этим значениям (ориентированный на значения и поведенческий).

Исторический обзор развития теории типов

Ключевые вехи развития теории типов и λ -исчисления:

- 1940-е: А. Черч — просто типизированное λ -исчисление ($\lambda_{\rightarrow}$).
- 1958/69: К. Гёдель — система T (примитивная рекурсия по натуральным), тотальные программы.
- 1968: Н. де Брёййн — Automath (ранний proof assistant).
- 1972 (Жирар), 1974 (Рейнольдс): система F (полиморфизм второго порядка).
- 1978: Милнер — полиморфизм в ML и идея главных типов.
- 1982: Дамас–Милнер — теорема о главной типовой схеме (Hindley–Milner).
- 1973/1984: П. Мартин-Лёф — зависимые типы и интуиционистская теория типов.
- 1985–88: Кокан, Юэ — Calculus of Constructions; 1986–: Isabelle/HOL; 1990–: Coq.
- 2010-е: Гомотопическая теория типов и унивалентные основания (Воеводский), кубическая ТТ.
- 2000–2020-е: прикладные ЯП с «богатыми» типами: ML/Haskell (НМ, ADT), Agda/Idris (зависимые типы).

- В более общем смысле, системы типов относятся и к математике, и к философии. В этом смысле системы типов были впервые использованы в 1900-х как способ избежать логических парадоксов (например, парадокса Рассела, 1902).
- В течение 20-го века типы превратились в стандартный инструмент логики, в частности, в теории доказательств:
 - теория типов Рассела (ramified theory of types, 1910)
 - простая теория типов Рамсея (simple theory of types, 1925)
 - конструктивная теория типов Мартина-Лёфа (constructive theory of types, 1973, 1984)
 - чистые системы типов Берарди (1988), Терлоу (1989) и Берендрегта (1992)
- Практическое применение: доказательные ассистенты и ИТС — HOL/Isabelle (простая ТТ), Coq/Lean (зависимые типы, CoC/Calculus of Inductive Constructions), Agda/Idris (MLTT).

- Система типов (по *TAPL*) — управляемый синтаксический метод доказательства отсутствия в программе определённых видов поведения посредством классификации выражений по видам вычисляемых значений.
- Системы типов можно рассматривать как статическую аппроксимацию поведения программы во время выполнения.
- Консервативность: гарантируют отсутствие ошибок заданного класса, но не их наличие, при этом некоторые корректные программы могут быть не типизируемы.

- Прикладная ветвь (языки программирования) часто не требует тотальности, т.к. большинство языков допускают частичные функции.
- Абстрактная ветвь связана с соответствием Карри—Ховарда—Ламбека и логикой доказательств.
- Различие в целях влияет на дизайн правил типизации и свойств (прогресс/сохранение, нормализация).

- 1 Мотивация и исторические перспективы теории типов
- 2 Просто типизированное λ -исчисление: синтаксис и правила типизации
- 3 Метатеоретические свойства и основные результаты

Просто типизированное λ -исчисление $\lambda_{\rightarrow}$

Теперь перейдём к формальному определению системы, которая решает указанные проблемы.

Определение (Тип)

Тип — это метка, которая присваивается терму и обладает некоторыми свойствами. Он является способом классификации термов.

Просто типизированное λ -исчисление обычно обозначается как **STLC** или $\lambda_{\rightarrow}$.

Определение (Множество типов)

Множество типов Θ из $\lambda_{\rightarrow}$ определяется индуктивно:

- 1 Переменные типа: $\alpha, \beta, \dots \in \Theta$
- 2 Типы функций (стрелочные типы): $\forall \sigma, \tau \in \Theta \Rightarrow (\sigma \rightarrow \tau) \in \Theta$

Соглашение (единая нотация): греческие буквы $\alpha, \beta, \gamma, \delta, \sigma, \tau$ обозначают переменные типов; латинские x, y, z — терм-переменные.

Примеры типов в $\lambda_{\rightarrow}$

Тип функции правоассоциативен, это отражает порядок принимаемых аргументов:

$$\sigma_1 \rightarrow \sigma_2 \rightarrow \cdots \rightarrow \sigma_n \equiv \sigma_1 \rightarrow (\sigma_2 \rightarrow (\cdots \rightarrow \sigma_n))$$

Примеры:

$\alpha \rightarrow \beta$ — функция от типа α к типу β

$\alpha \rightarrow \beta \rightarrow \gamma$ — функция двух аргументов (каррирование)

$(\alpha \rightarrow \beta) \rightarrow \gamma$ — функция высшего порядка

$(\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$ — функция, принимающая функцию и значение

Интуиция: $\sigma \rightarrow \tau$ читается как «функция, которая принимает аргумент типа σ и возвращает значение типа τ ».

Определение (Предтермы)

M, N	$::=$	x	переменная
		$M N$	аппликация
		$\lambda x. M$	абстракция (стиль Карри)
		$\lambda x^\sigma. M$	абстракция (стиль Черча)

Эти правила задают *синтаксис термов* независимо от типовой информации: сами предтермы могут быть нетипизированными, а типы присваиваются отдельно правилами типизации.

Определение (Объявление)

Утверждение о типе переменной, например $x:\sigma$.

Определение (Контекст)

Контекст — конечное отображение $\Gamma = \{x_i:\sigma_i\}_{i=1..n}$ на попарно различных переменных.

Определение (Домен контекста)

Домен: $\text{dom}(\Gamma) = \{x_1, \dots, x_n\}$, а для переменной x запись $\Gamma(x)$ — её тип в Γ (если определён).

Определение (Сужение контекста)

Сужение $\Gamma \upharpoonright V'$ на множество переменных $V' \subseteq \text{dom}(\Gamma)$: это контекст $\{x:\Gamma(x) \mid x \in V'\}$.

- Запись $\Gamma:V$ означает, что $\text{dom}(\Gamma) \subseteq V$.

Определение (Утверждение о типизации)

Запись $\Gamma \vdash M : \tau$ означает, что при предположениях из контекста Γ терм M имеет тип τ по правилам типизации.

- Читается: «в контексте Γ терм M имеет тип τ ».
- Эквивалентно существованию дерева вывода по правилам типизации.
- Если контекст не указан, подразумевается пустой контекст $\emptyset$.
- Пример: при $\Gamma = \{x:\alpha, f:\alpha \rightarrow \beta\}$ верно $\Gamma \vdash f\ x : \beta$.

Структурные свойства типизации

Пусть $\Gamma \vdash M : \sigma$.

Структурные свойства

- 1 **Ослабление:** если $\Delta \supseteq \Gamma$ и $\text{dom}(\Delta)$ содержит $\text{dom}(\Gamma)$ без конфликтов типов, то $\Delta \vdash M : \sigma$.
- 2 **Покрытие свободных переменных:** $\text{FV}(M) \subseteq \text{dom}(\Gamma)$.
- 3 **Сужение до свободных:** $\Gamma \upharpoonright \text{FV}(M) \vdash M : \sigma$.

Идеи доказательств: индукция по выводу, шаги $\text{Var}/\rightarrow$ $E/\rightarrow$ I проверяются напрямую, используя свойства добавления/удаления объявлений для нерелевантных переменных.

Принципы типизации термов

Пусть контекст Γ сопоставляет переменным их типы.
Правила вывода формируют утверждения вида $\Gamma \vdash M : \tau$.

Переменная (Var)

Если $x:\tau \in \Gamma$, то $\Gamma \vdash x : \tau$.
Пример: при $\Gamma = \{x:\alpha\}$
верно $\Gamma \vdash x : \alpha$.

Аппликация (App)

Если $\Gamma \vdash M : \sigma \rightarrow \tau$ и
 $\Gamma \vdash N : \sigma$, то $\Gamma \vdash MN : \tau$.
Пример: при
 $\Gamma = \{f:\alpha \rightarrow \beta, x:\alpha\}$ верно
 $\Gamma \vdash f x : \beta$.

Абстракция (Abs)

Если $\Gamma, x:\sigma \vdash M : \tau$, то
 $\Gamma \vdash \lambda x.M : \sigma \rightarrow \tau$.
Пример: из $x:\alpha \vdash x : \alpha$
получаем $\emptyset \vdash \lambda x.x : \alpha \rightarrow \alpha$.

Правила типизации в натуральном выводе

Базовые правила можно записать в стиле натурального вывода:

$$\frac{x:\sigma \in \Gamma}{\Gamma \vdash x : \sigma} \text{Var} \qquad \frac{\Gamma \vdash M : \sigma \rightarrow \tau \quad \Gamma \vdash N : \sigma}{\Gamma \vdash MN : \tau} \rightarrow E \qquad \frac{\Gamma, x:\sigma \vdash M : \tau}{\Gamma \vdash \lambda x.M : \sigma \rightarrow \tau} \rightarrow I$$

- Сверху — посылки, снизу — заключение.
- Правила: Var (переменная), $\rightarrow E$ (элиминация), $\rightarrow I$ (введение абстракции).

Стили типизации: Карри против Черча

- Стил Карри: аннотации типов отсутствуют, типы выводятся из контекста.
- Стил Черча: у абстракции указана аннотация $\lambda x:\sigma.M$; она используется в правиле Abs.

Обозначение в стиле Черча:

$$\frac{\Gamma, x:\sigma \vdash M : \tau}{\Gamma \vdash \lambda x^\sigma.M : \sigma \rightarrow \tau} \rightarrow I_{\text{Church}}$$

Пример построения дерева вывода

Пример

Пусть $x:\gamma$, $y:\gamma \rightarrow \beta$, $z:\beta \rightarrow \delta$ в Γ . Тогда

$$\Gamma \vdash z(y\ x) : \delta$$

Дерево вывода:

$$\frac{\frac{\overline{\Gamma \vdash z : \beta \rightarrow \delta} \text{ Var} \quad \frac{\frac{\overline{\Gamma \vdash y : \gamma \rightarrow \beta} \text{ Var} \quad \overline{\Gamma \vdash x : \gamma} \text{ Var}}{\Gamma \vdash y\ x : \beta} \rightarrow E}{\Gamma \vdash z(y\ x) : \delta} \rightarrow E$$

Пример: композиция в стиле Карри

Хотим вывести тип $\Gamma \vdash \lambda x y. x (y x) : (\beta \rightarrow \delta) \rightarrow ((\beta \rightarrow \delta) \rightarrow \beta) \rightarrow \delta$.

Промежуточные гипотезы: $x:\beta \rightarrow \delta$, $y:(\beta \rightarrow \delta) \rightarrow \beta$.

Тогда $y x : \beta$ и $x (y x) : \delta$.

Дерево вывода:

$$\frac{\frac{\frac{\dots \vdash x : \beta \rightarrow \delta \quad \text{Var}}{\dots \vdash y x : \beta} \quad \frac{\frac{\dots \vdash y : (\beta \rightarrow \delta) \rightarrow \beta \quad \text{Var}}{\dots \vdash x : \beta \rightarrow \delta} \quad \text{Var}}{\dots \vdash y x : \beta} \quad \rightarrow E}{\Gamma, x:\beta \rightarrow \delta, y:(\beta \rightarrow \delta) \rightarrow \beta \vdash x (y x) : \delta} \rightarrow E}{\Gamma, x:\beta \rightarrow \delta \vdash \lambda y. x (y x) : ((\beta \rightarrow \delta) \rightarrow \beta) \rightarrow \delta} \rightarrow I}{\Gamma \vdash \lambda x y. x (y x) : (\beta \rightarrow \delta) \rightarrow ((\beta \rightarrow \delta) \rightarrow \beta) \rightarrow \delta} \rightarrow I$$

Тот же пример в стиле Черча

Аннотируем параметры и получаем тот же тип:

$$\Gamma \vdash \lambda x^{\beta \rightarrow \delta} y^{(\beta \rightarrow \delta) \rightarrow \beta}. x (y x) : (\beta \rightarrow \delta) \rightarrow ((\beta \rightarrow \delta) \rightarrow \beta) \rightarrow \delta$$

Дерево вывода строится аналогично, но правило Abs использует аннотацию $y\ x$ и y .

- ❶ Well-typedness / Type assignment: для данного M найти τ и/или Γ , такие что $\Gamma \vdash M : \tau$.
- ❷ Type checking: по Γ , M и τ решить, верно ли $\Gamma \vdash M : \tau$.
- ❸ Type inhabitation: по Γ и τ решить, существует ли M такое, что $\Gamma \vdash M : \tau$.

Задача присваивания типов

Формулировка: найти тип τ (и, при необходимости, контекст Γ), такой что $\Gamma \vdash M : \tau$.

Замечания:

- В стиле Карри это задача выведения типов; без полиморфизма возможна *главная* (наиболее общая) типизация.
- В стиле Черча аннотации сужают поиск: часть типов задаётся в самом терме и упрощает вывод.
- В $\lambda_{\rightarrow}$ (без подтипов/равенства сложных типов) задача решается через унификацию уравнений на типы.
- В ML/Haskell (Hindley–Milner) используется алгоритм W (или эквивалентные) с генерацией ограничений и унификацией.
- Ограничения: наличие подтипов, зависимых типов или сложного равенства типов может сделать задачу существенно труднее или неразрешимой.

Пример: для $M \equiv \lambda f x. f x$ находим типы $f:\alpha \rightarrow \beta$, $x:\alpha$, тогда $M:(\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$ (главный тип).

Унификация 101: occurs-check

Цель: решить систему уравнений на типы (стрелочные типы) или сообщить «нет решения».
Базовые шаги (алгоритм Робинсона):

$$\frac{\sigma_1 = \tau_1 \quad \sigma_2 = \tau_2}{\sigma_1 \rightarrow \sigma_2 = \tau_1 \rightarrow \tau_2} \text{ dec} \quad \frac{\alpha \notin \text{FV}(\tau)}{\alpha = \tau} \text{ var} \Rightarrow [\alpha := \tau]$$

Дополнительно: симметрия/ориентация уравнений; удаление тождеств $\tau = \tau$.

Occurs-check: если $\alpha \in \text{FV}(\tau)$, то унификация $\alpha = \tau$ невозможна (предотвращает «бесконечные типы»).

Пример: требование типизуемости `Id Id` породило бы уравнение $\alpha = \alpha \rightarrow \alpha$ — провал по occurs-check.

Используется в алгоритме W: генерация ограничений + унификация с occurs-check.

Задача проверки типов

Дано: Γ, M, σ . Задача: проверить выполнимость $\Gamma \vdash M : \sigma$.

В просто типизированном $\lambda_{\rightarrow}$ задача разрешима алгоритмом рекурсивного спуска по структуре терма с проверкой правил $\text{Var} / \rightarrow E / \rightarrow I$.

- Идея: проходим по M ; для переменных сверяем тип в Γ ; для аппликации сверяем стрелочный тип функции; для абстракции проверяем тело под расширенным контекстом.
- При стиле Черча сверяем декларированный тип параметра $x:\sigma$ с ожидаемым; при Карри — требуются вывод и/или сравнение типов.
- Эквивалентность типов в $\lambda_{\rightarrow}$ тривиальна (структурная); при добавлении сокращений/ η -равенства/подтипов проверка усложняется.
- Практика: в компиляторах используется комбинированный «синтез/проверка» (bidirectional typing), чтобы локализовать вывод и улучшить сообщения об ошибках.

Двунаправленная типизация: синтез/проверка

Два режима вывода типов: *синтез* «снизу вверх» и *проверка* «сверху вниз». Это локализует вывод и делает ошибки короче и точнее.

$$\Gamma \vdash e \Rightarrow A \quad (\text{синтез}), \quad \Gamma \vdash e \Leftarrow A \quad (\text{проверка})$$

Где что применяется (в $\lambda_{\rightarrow}$ и практических ЯП):

- **Синтезируют:** переменные/константы, аппликации, проекции — форма сама даёт тип-результат.
- **Проверяются:** интродукции (λ , пары, конструкторы данных) — требуют ожидаемого типа сверху.
- **Аннотация** у λ переводит её из проверки в синтез.

Минимальные правила для $\lambda_{\rightarrow}$:

$$\begin{array}{c} \frac{x:\tau \in \Gamma}{\Gamma \vdash x \Rightarrow \tau} \text{Var} \qquad \frac{\Gamma \vdash M \Rightarrow \sigma \rightarrow \tau \quad \Gamma \vdash N \Leftarrow \sigma}{\Gamma \vdash M N \Rightarrow \tau} \rightarrow E \\[10pt] \frac{\Gamma, x:\sigma \vdash M \Leftarrow \tau}{\Gamma \vdash \lambda x.M \Leftarrow \sigma \rightarrow \tau} \rightarrow I \qquad \frac{\Gamma, x:\sigma \vdash M \Leftarrow \tau}{\Gamma \vdash \lambda x^\sigma.M \Rightarrow \sigma \rightarrow \tau} \rightarrow I^{\text{ann}} \end{array}$$

Пример (TypeScript/Haskell):

- TS: `const inc: (x: number) => number = x => x + 1`. Ожидаемый тип сверху *проверяет* тело (возврат `number`); внутри `+` и `1` *синтезируют* числа. Также: `[1,2].map(x => x + 1)` — тип `x` приходит сверху из `map`, а выражение синтезирует `number`.
- Haskell: `foldr (x acc -> x + acc) 0 xs`. По сигнатуре `foldr` ожидаемый тип аргумента сверху *проверяет* лямбду, а применения и литералы *синтезируют* типы снизу.

Задача обитаемости типов

Дано: Γ , тип σ . Задача: существует ли M с $\Gamma \vdash M : \sigma$? Эквивалентна доказуемости формулы импликационной интуиционистской логики (через Карри—Ховарда).

- В чистом импликационном фрагменте (только $\rightarrow$) задача *разрешима* и PSPACE-полна (Statman, 1979).
- Поиск ведётся как построение вывода «снизу вверх» (proof search): попытка применить $\rightarrow I$ и $\rightarrow E$ к цели/подцелям.
- Интуитивно: чтобы обитать $\sigma \rightarrow \tau$, достаточно уметь построить τ при добавлении гипотезы $x:\sigma$ в контекст.
- Примеры: $\alpha \rightarrow \alpha$ обитаем (терм $\lambda x.x$); $(\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \beta$ обитаем (терм композиции/применения); α без предпосылок — необитаем.

1) Обитаемость Тип $(\alpha \rightarrow \beta \rightarrow \alpha) \rightarrow \beta \rightarrow \alpha$ — обитаем? **Нет (в чистом $\lambda_{\rightarrow}$).**

Почему: при $f:\alpha \rightarrow \beta \rightarrow \alpha$ и $y:\beta$ получить α можно лишь как $f\ u\ y$ с $u:\alpha$, но значения типа α из контекста нет. Попытка подставить $u \equiv K$ с $K:\alpha \rightarrow \beta \rightarrow \alpha$ требует унификации $\alpha \cong \alpha \rightarrow \beta \rightarrow \alpha$ (occurs-check) — противоречие.

2) Контрпример к subject expansion

$M \equiv (\lambda x. x\ x)\text{Id} \rightarrow_{\beta} \text{Id}\text{Id} \rightarrow_{\beta} \text{Id} \equiv N.$ N типизуем, M — нет.

3) Дерево вывода для $\lambda f x. f\ (f\ x)$ с типом $(\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha.$

Дерево вывода: $\lambda f x. f (f x)$

Цель: $\Gamma \vdash \lambda f x. f (f x) : (\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$.

$$\frac{\frac{\frac{\overline{\dots \vdash f : \alpha \rightarrow \alpha}}{\dots \vdash f : \alpha \rightarrow \alpha} \text{Var} \quad \frac{\frac{\overline{\dots \vdash f : \alpha \rightarrow \alpha}}{\dots \vdash f : \alpha \rightarrow \alpha} \text{Var} \quad \frac{\overline{\dots \vdash x : \alpha}}{\dots \vdash x : \alpha} \text{Var}}{\dots \vdash f x : \alpha} \rightarrow E}{\Gamma, f : \alpha \rightarrow \alpha, x : \alpha \vdash f (f x) : \alpha} \rightarrow E}{\Gamma, f : \alpha \rightarrow \alpha \vdash \lambda x. f (f x) : \alpha \rightarrow \alpha} \rightarrow I}{\Gamma \vdash \lambda f x. f (f x) : (\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha} \rightarrow I$$

Таким образом, $\lambda f x. f (f x)$ имеет тип $(\alpha \rightarrow \alpha) \rightarrow \alpha \rightarrow \alpha$.

Полное дерево вывода композиции

$$\begin{array}{c}
 \frac{\overline{\dots \vdash f : \beta \rightarrow \delta} \text{ Var}}{\Gamma, f : \beta \rightarrow \delta, g : \alpha \rightarrow \beta, x : \alpha \vdash f(gx) : \delta} \text{ Var} \quad \frac{\overline{\dots \vdash g : \alpha \rightarrow \beta} \text{ Var} \quad \overline{\dots \vdash x : \alpha} \text{ Var}}{\dots \vdash gx : \beta} \text{ Var} \\
 \frac{\overline{\dots \vdash f : \beta \rightarrow \delta} \text{ Var} \quad \overline{\dots \vdash gx : \beta} \text{ Var}}{\Gamma, f : \beta \rightarrow \delta, g : \alpha \rightarrow \beta, x : \alpha \vdash f(gx) : \delta} \rightarrow E \\
 \frac{\Gamma, f : \beta \rightarrow \delta, g : \alpha \rightarrow \beta, x : \alpha \vdash f(gx) : \delta}{\Gamma, f : \beta \rightarrow \delta, g : \alpha \rightarrow \beta \vdash \lambda x. f(gx) : \alpha \rightarrow \delta} \rightarrow I \\
 \frac{\Gamma, f : \beta \rightarrow \delta, g : \alpha \rightarrow \beta \vdash \lambda x. f(gx) : \alpha \rightarrow \delta}{\Gamma, f : \beta \rightarrow \delta \vdash \lambda gx. f(gx) : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \delta} \rightarrow I \\
 \frac{\Gamma, f : \beta \rightarrow \delta \vdash \lambda gx. f(gx) : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \delta}{\Gamma \vdash \lambda fgx. f(gx) : (\beta \rightarrow \delta) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \delta} \rightarrow I
 \end{array}$$

Пометки: Var — аксиомы, $\rightarrow E$ — элиминация, $\rightarrow I$ — интродукция импликации.

Альтернативная запись: флаговая нотация

Идея: записывать вывод построчно с **флагами-допущениями** и ссылками на правила.

- Слева — активные допущения: например, $\boxed{1: f:\beta \rightarrow \delta}$, $\boxed{2: g:\alpha \rightarrow \beta}$, $\boxed{3: x:\alpha}$.
- Каждая строка имеет номер и суждение, справа — указание правила и номеров строк-оснований: например, $(\rightarrow E\ 2, 3)$.
- Снятие допущения при интродукции стрелки помечаем $(\rightarrow I\ n)$, где n — номер строки, после которой закрываем флаг.

Композиция во флаговой нотации

1: $f:\beta \rightarrow \delta$
2: $g:\alpha \rightarrow \beta$
3: $x:\alpha$

- 4. $g\ x : \beta$ ($\rightarrow E$ 2, 3)
- 5. $f\ (g\ x) : \delta$ ($\rightarrow E$ 1, 4)
- 6. $\lambda x. f\ (g\ x) : \alpha \rightarrow \delta$ ($\rightarrow I$ закрываем 3)
- 7. $\lambda gx. f\ (g\ x) : (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \delta$ ($\rightarrow I$ закрываем 2)
- 8. $\lambda fgx. f\ (g\ x) : (\beta \rightarrow \delta) \rightarrow (\alpha \rightarrow \beta) \rightarrow \alpha \rightarrow \delta$ ($\rightarrow I$ закрываем 1)

Где «закрываем k » означает снятие флага-допущения с меткой k .

Философия типизации: внутренние и внешние типы

Стиль Черча (intrinsic): типы *встроены в семантику* языка. Значение программы зависит от типовой информации; многое решается на этапе компиляции.

Примеры: Haskell и др. со статической обязательной типизацией.

Стиль Карри (extrinsic): типы — *внешняя надстройка* над уже заданной семантикой; программу можно выполнять и без проверок. Можно применять *несколько* систем типов к одному коду.

Примеры: Clojure с 'core.typed', внешние анализаторы.

Источник: [Eric Normand, Church vs Curry Types](#).

Перспектива Черча

типизированные программы

«нетипизированные»
как один тип

Перспектива Карри

динамические программы

типизированное
подмножество

- **Черч/intrinsic**: математически обоснованная безопасность, мощные инварианты на этапе компиляции.
- **Карри/extrinsic**: можно накладывать *несколько* систем типов; анализ без изменения языка/рантайма.
- **Мосты**: *gradual typing*, *blame calculus*, сочетание intrinsic/extrinsic в одном языке; пример внешнего усиления — *Liquid Haskell*.

Лемма об инверсии правил типизации

Лемма (Лемма об инверсии правил типизации)

Если выводимо $\Gamma \vdash M : \tau$, то верны обратные условия к правилам:

- *Var*: из $\Gamma \vdash x : \sigma$ следует $x:\sigma \in \Gamma$.
- $\rightarrow E$: из $\Gamma \vdash M N : \tau$ следует $\exists \sigma [\Gamma \vdash M : \sigma \rightarrow \tau \wedge \Gamma \vdash N : \sigma]$.
- $\rightarrow I$ (Карри): из $\Gamma \vdash \lambda x.M : \tau$ следует $\exists \sigma, \rho [\Gamma, x:\sigma \vdash M : \rho \wedge \tau \equiv \sigma \rightarrow \rho]$.
- $\rightarrow I$ (Черч): из $\Gamma \vdash \lambda x:\sigma.M : \tau$ следует $\exists \rho [\Gamma, x:\sigma \vdash M : \rho \wedge \tau \equiv \sigma \rightarrow \rho]$.

Доказательство.

Индукция по последнему правилу вывода $\Gamma \vdash M : \tau$:

- Если последнее правило — *Var*, то $M \equiv x$ и по правилу $x : \tau \in \Gamma$.
- Если последнее правило — $\rightarrow E$, то $M \equiv M_1 M_2$, и существуют выводы $\Gamma \vdash M_1 : \sigma \rightarrow \tau$ и $\Gamma \vdash M_2 : \sigma$ для некоторого σ .
- Если последнее правило — $\rightarrow I$, то $M \equiv \lambda x.P$ (Карри) или $M \equiv \lambda x : \sigma.P$ (Черч), и существует вывод $\Gamma, x : \sigma \vdash P : \rho$ с $\tau \equiv \sigma \rightarrow \rho$.

Типизуемость подтермов

Лемма (Лемма о типизуемости подтермов)

Если $\Gamma \vdash M : \sigma$ и M' — подтерм M , то существуют тип σ' и контекст Γ' , такие, что $\Gamma' \vdash M' : \sigma'$.

Доказательство.

- База: $M \equiv x$ — следует из инверсии Var.
- Случай аппликации: инверсия $\rightarrow E$ даёт типы для функции и аргумента.
- Случай абстракции: инверсия $\rightarrow I$ даёт типизуемость тела при добавлении $x:\sigma$ в контекст.



Следствие: каждый шаг корректного вывода соответствует типизуемому подтерму исходного терма. Репарка: для M' используется *подходящий* контекст (сужение/расширение исходного), обычно $\Gamma' = (\Gamma \upharpoonright \text{FV}(M'))$ или его расширение на аннотированные параметры.

Теорема

Если $\Gamma \vdash M : \sigma$ и $\Gamma \vdash M : \tau$, то $\sigma \equiv \tau$.

Доказательство.

- Var: из инверсии имеем $x:\sigma \in \Gamma$ и $x:\tau \in \Gamma$; при хорошо сформированном контексте тип переменной единственен, значит $\sigma \equiv \tau$.
- $\rightarrow E$: из инверсии получаем $\Gamma \vdash M_1 : \rho \rightarrow \sigma$, $\Gamma \vdash M_2 : \rho$ и $\Gamma \vdash M_1 : \rho' \rightarrow \tau$, $\Gamma \vdash M_2 : \rho'$. По ИП $\rho \equiv \rho'$ и $\rho \rightarrow \sigma \equiv \rho' \rightarrow \tau$, откуда $\sigma \equiv \tau$.
- $\rightarrow I$: из инверсии имеем $\Gamma, x:\rho \vdash P : \theta$ и $\Gamma, x:\rho' \vdash P : \theta'$, а также $\sigma \equiv \rho \rightarrow \theta$, $\tau \equiv \rho' \rightarrow \theta'$. По ИП для тела P (и корректности расширения контекста) $\rho \equiv \rho'$ и $\theta \equiv \theta'$, следовательно $\sigma \equiv \tau$.



Следствия из единственности типов:

- Детерминированность проверки типов: если вывод существует, целевой тип единственен.
- В стиле Черча у $\lambda x:\sigma. M$ тип всегда имеет вид $\sigma \rightarrow \tau$ и он единственный.
- В стиле Карри существует *главная типизация*: для $\Gamma \vdash M : \sigma$ есть наиболее общий тип τ , такой что $\sigma = S(\tau)$ для некоторой подстановки S (пример: $\lambda x.x : \alpha \rightarrow \alpha$; частные случаи — $\text{Bool} \rightarrow \text{Bool}$, $\text{Nat} \rightarrow \text{Nat}$ и т.п.).

Запрет самоприменения в типизированном исчислении

Тезис

Не существует контекста Γ и типа τ , таких что $\Gamma \vdash x x : \tau$.

Доказательство.

Пусть, для противоречия, $\Gamma \vdash x x : \tau$. По инверсии для $\rightarrow E$ существуют σ такие, что $\Gamma \vdash x : \sigma \rightarrow \tau$ и $\Gamma \vdash x : \sigma$. По лемме об единственности типов при фиксированном Γ тип переменной x единственен; обозначим его через T . Тогда имеем уравнение типов $T \equiv T \rightarrow \tau$.

Определим меру на типах: $|\alpha| = 0$, $|\rho \rightarrow \zeta| = 1 + |\rho| + |\zeta|$. Из $T \equiv T \rightarrow \tau$ следует $|T| = 1 + |T| + |\tau|$, что невозможно. Противоречие. Следовательно, $x x$ нетипизуемо в $\lambda_{\rightarrow}$. □

Следствие: нетипизируемость комбинатора Ω

Классический нетерминирующий комбинатор $\Omega = (\lambda x. x x) (\lambda x. x x)$ не типизируем в $\lambda_{\rightarrow}$.

- Внутренний терм $x x$ не типизируем (предыдущий кадр) — значит ни одно его вхождение не может получить тип.
- Таким образом, ни одна часть аппликации в Ω не типизируема — типизации Ω не существует.

Ключевой результат: простая типизация запрещает самоприменение, тем самым исключая общие фиксированные точки и бесконечные вычисления.

Следствие: все типизируемые термы в $\lambda_{\rightarrow}$ *сильно нормализуются* — их вычисление всегда завершается.

- 1 Мотивация и исторические перспективы теории типов
- 2 Просто типизированное λ -исчисление: синтаксис и правила типизации
- 3 **Метатеоретические свойства и основные результаты**

Подстановка типов: определения

Перейдём к изучению формальных свойств нашей системы типов. Пусть $\sigma, \zeta \in \Theta$ и $\alpha \in \Theta$ — переменная типа. Обозначаем $\sigma[\alpha := \zeta]$ подстановку ζ вместо свободных вхождений α в σ .

$$\alpha[\alpha := \zeta] \equiv \zeta,$$

$$\beta[\alpha := \zeta] \equiv \beta \quad (\beta \neq \alpha),$$

$$(\rho \rightarrow \tau)[\alpha := \zeta] \equiv \rho[\alpha := \zeta] \rightarrow \tau[\alpha := \zeta].$$

Для контекстов используем поточечное действие: если $\Gamma \equiv x_1:\tau_1, \dots, x_n:\tau_n$, то

$$\Gamma[\alpha := \zeta] \equiv x_1:\tau_1[\alpha := \zeta], \dots, x_n:\tau_n[\alpha := \zeta].$$

Подстановка типов: согласованность

Лемма (Согласованность типов с подстановкой)

Если $\Gamma \vdash M : \sigma$, то $\Gamma[\alpha := \zeta] \vdash M : \sigma[\alpha := \zeta]$.

Доказательство.

Индукция по выводу типизации:

- **Var**: если $\Gamma \vdash x : \tau$ и $x:\tau \in \Gamma$, то $x:\tau[\alpha := \zeta] \in \Gamma[\alpha := \zeta]$.
- **App**: применяем И.П. к обеим посылкам правила $\rightarrow E$.
- **Abs**: применяем И.П. к посылке правила $\rightarrow I$ с расширенным контекстом.



Замечание: в некотором смысле подстановка типов здесь — *мета-операция* (т.к. в $\lambda_{\rightarrow}$ нет кванторов по типам как в System F).

Подстановка терма

Лемма (О подстановке терма)

Если $\Gamma, x:\sigma \vdash M : \tau$ и $\Gamma \vdash N : \sigma$, то $\Gamma \vdash M[x := N] : \tau$.

Доказательство.

Индукция по структуре терма M :

- **Случай $M \equiv x$:** Тогда $M[x := N] \equiv N$. Из $\Gamma, x : \sigma \vdash x : \tau$ по инверсии получаем $\tau \equiv \sigma$. Значит, $\Gamma \vdash N : \sigma \equiv \tau$.
- **Случай $M \equiv y$ ($y \neq x$):** Тогда $M[x := N] \equiv y$. Из $\Gamma, x : \sigma \vdash y : \tau$ получаем $y : \tau \in \Gamma$, откуда $\Gamma \vdash y : \tau$.
- **Случай $M \equiv M_1 M_2$:** По И.П. $\Gamma \vdash M_1[x := N] : \rho \rightarrow \tau$ и $\Gamma \vdash M_2[x := N] : \rho$. По правилу $\rightarrow E$ получаем $\Gamma \vdash (M_1 M_2)[x := N] : \tau$.
- **Случай $M \equiv \lambda y. P$ ($y \neq x, y \notin \text{FV}(N)$):** По И.П. $\Gamma, y : \rho \vdash P[x := N] : \zeta$. По правилу $\rightarrow I$ получаем $\Gamma \vdash \lambda y. (P[x := N]) : \rho \rightarrow \zeta$.

Теорема о сохранении типа при редукции

Теорема

Если $M \rightarrow_{\beta} N$ и $\Gamma \vdash M : \sigma$, то $\Gamma \vdash N : \sigma$.

Доказательство.

Индукция по отношению $\rightarrow_{\beta}$. Достаточно рассмотреть один шаг β -редукции.

Базовый случай: Пусть $M \equiv (\lambda x.P)Q \rightarrow_{\beta} P[x := Q] \equiv N$.

- Из $\Gamma \vdash (\lambda x.P)Q : \sigma$ по лемме об инверсии для $\rightarrow E$ получаем существование типа ζ такого, что $\Gamma \vdash \lambda x.P : \zeta \rightarrow \sigma$ и $\Gamma \vdash Q : \zeta$.
- Из $\Gamma \vdash \lambda x.P : \zeta \rightarrow \sigma$ по лемме об инверсии для $\rightarrow I$ получаем $\Gamma, x : \zeta \vdash P : \sigma$.
- По лемме о подстановке терма: $\Gamma \vdash P[x := Q] : \sigma$.

Индуктивный случай: Если $M \rightarrow_{\beta} N$ в подтерме, применяем индукционную гипотезу и правила типизации для восстановления типа всего терма. □

Интуиция: тип — инвариант вычислений; редукция не меняет объявленный интерфейс.

Предпосылки: леммы инверсии, корректность подстановки, структурная индукция по редукции.

Пример: если $\Gamma \vdash (\lambda x:\alpha. x) Q : \alpha$, то один шаг β даёт Q того же типа α .

Сохранение: обратная импликация неверна

Обратное утверждение в общем неверно: из $\Gamma \vdash N : \sigma$ и $M \rightarrow_\beta N$ не следует $\Gamma \vdash M : \sigma$.
Контрпример: возьмём $\text{Id} \equiv \lambda x.x : \alpha \rightarrow \alpha$.

$$M \equiv (\lambda x.x x) \text{Id} \rightarrow_\beta \text{Id Id} \rightarrow_\beta \text{Id} \equiv N.$$

Но M нетипизуем (содержит самоприменение), тогда как N типизуем с типом $\alpha \rightarrow \alpha$.
Почему Id Id нетипизуемо? Не проходит occurs-check: уравнение типов $\alpha \cong \alpha \rightarrow \alpha$ требует α содержать себя (циклически), что невозможно в простых типах.

Канонические формы и теорема прогресса

Определение (Значения в чистом $\lambda_{\rightarrow}$)

Значения определяются как $V ::= \lambda x.M$.

Лемма (Канонические формы)

Если $\emptyset \vdash V : \sigma \rightarrow \tau$ и V — значение, то $V \equiv \lambda x.M$.

Теорема (Прогресс)

Если $\emptyset \vdash M : \tau$, то либо M — значение, либо $\exists N. M \rightarrow_{\beta} N$.

Идея доказательства: индукция по выводу для $\emptyset \vdash M : \tau$; случай аппликации опирается на лемму о канонических формах.

Интерпретация: типизуемый замкнутый терм либо уже «готов» (значение), либо делает шаг; тупиковых ситуаций нет.

Канонические формы (расширено):

- Если $\emptyset \vdash V : \sigma \rightarrow \tau$ и V — значение, то $V \equiv \lambda x.M$.
- Если $\emptyset \vdash V : \alpha$ и V — значение базового типа, то V имеет форму константы/литерала соответствующего типа (в расширениях с базисом).

Примеры:

- $(\lambda f x.f\ x) (\lambda y.y)$ — не значение, но редуцируется за один шаг.
- $\lambda x.x$ — значение, прогресс выполнен тривиально.

Safety = Progress + Preservation

Определение безопасности

Система типов *безопасна*, если замкнутые типизуемые термы не попадают в тупик вычисления: на каждом шаге либо значение, либо существует редукция.

- **Progress (Прогресс)**: типизуемый замкнутый терм либо значение, либо делает шаг.
- **Preservation (Сохранение)**: если $\Gamma \vdash M : \tau$ и $M \rightarrow N$, то $\Gamma \vdash N : \tau$.

Вместе эти свойства гарантируют: вычисление типизуемых программ не «ломает» типы и не приводит к тупику.

Следствия для ЯП: отсутствие класса ошибок «stuck» во время выполнения, более сильные инварианты для оптимизаций и рефакторинга.

Интерпретация типов в прикладных ЯП: типы задают *контракты* функций.

Примеры из разных языков:

- **Haskell**: `map :: (a -> b) -> [a] -> [b]`
- **OCaml**: `let compose f g x = f (g x) (* ('b -> 'c) -> ('a -> 'b) -> 'a -> 'c *)`
- **TypeScript**: `const identity = <T>(x: T): T => x`
- **Rust**: `fn apply<F, T, U>(f: F, x: T) -> U where F: Fn(T) -> U`

Свойство сохранения типа гарантирует: вычисления не меняют «контракт» функции.

Почему рекурсия есть, хотя Y нетипизуем?

В $\lambda_{\rightarrow}$ комбинатор Y нетипизуем из-за самоприменения (occurs-check). Тем не менее, практические ЯП предоставляют рекурсию как *синтаксическую конструкцию* с собственной типовой дисциплиной:

- OCaml: `let rec f x = ...` — правило типизации ослабляет требование на терминальность, но проверяет согласованность типов f в теле.
- Haskell: определение через связывание по уравнениям и ленивость; рекурсия разрешена семантикой (фиксированные точки в доменной модели).
- **Итог:** В практических ЯП рекурсия — это синтаксическая конструкция (`let rec`, `fix`) с отдельной типовой дисциплиной, она не выводится из $\lambda_{\rightarrow}$.

Просто типизированное λ -исчисление — фундамент современной теории типов:

- **Полиморфные системы:** System F , Hindley—Milner (ML, Haskell)
- **Зависимые типы:** Agda, Idris, Lean, Coq
- **Субструктурные типы:** Rust (ownership), Linear Haskell
- **Эффектные системы:** Koka, Eff, algebraic effects
- **Градуальная типизация:** TypeScript, Typed Racket

Принципы остаются неизменными:

- Статическая проверка корректности
- Предотвращение ошибок времени выполнения
- Документирование интерфейсов
- Оптимизация компилятора

Заключение: какие недостатки решаются в $\lambda_{\rightarrow}$

- **Самоприменение** $x\ x$ станет невозможным — переменная не может одновременно быть функцией и аргументом
- **Комбинатор неподвижной точки** Y станет нетипизируемым
- **Все типизируемые термы** будут иметь нормальную форму (сильная нормализация)
- **Логическая непротиворечивость** через соответствие Карри—Ховарда

Проблема: потеря выразительности — некоторые «хорошие» программы тоже станут нетипизируемыми.

Компромисс: безопасность против полноты по Тьюрингу.

Заключение: что мы изучили

- **Мотивация:** просто типизированное λ -исчисление решает проблемы нетипизированного исчисления (самоприменение, нетерминация).
- **Синтаксис:** типы $\sigma ::= \alpha \mid \sigma \rightarrow \tau$; правила $\text{Var}, \rightarrow E, \rightarrow I$.
- **Стили:** Карри (вывод типов) vs Черч (аннотированные типы).
- **Алгоритмы:** присваивание типов, проверка типов, обитаемость типов.
- **Метатеория:** инверсия, подстановка, сохранение типа при β -редукции.
- **Ограничения:** запрет самоприменения исключает общую рекурсию и Ω .

Следующие шаги: полиморфизм (система F), зависимые типы, практические системы типов.